Low Level Design Document

Introduction

This Low Level Design (LLD) document outlines the core components and implementation details for **MedBotX - Advanced Medical Chatbot with Memory**. MedBotX is a healthcare chatbot built using Python, FastAPI, LangChain, and OpenAI, designed to answer patient queries and maintain both temporary and permanent conversational memory.

---

1. System Components

Component	Description	Key Responsibilities
API Server	FastAPI-based REST API	Handle HTTP requests, route to services
Chatbot Engine	LangChain + OpenAI integration	Generate responses, manage context
Memory Manager	Handles session (temp) & persistent (perm) memory	Store/retrieve conversation history
Data Store	Persistent storage (e.g., SQLite, Redis, or file-based)	Store permanent memory, user data
Auth Module	(Optional) User authentication/session management	Secure endpoints, manage user sessions

---

2. Class/Interface Overview

Class/Interface	Description	Key Methods/Attributes
`ChatbotService`	Orchestrates chat flow	`get_response()` , `update_memory()`
`MemoryManager`	Manages temp & perm memory	`get_memory()` , `save_memory()`
`OpenAIClient`	Handles OpenAI API calls	`generate_response(prompt)`
`UserSession`	Represents user session/context	`session_id` , `user_id` , `history`
`PersistentStore`	Interface to data storage	`save()` , `load()` , `delete()`

Relationships:

- `ChatbotService` uses `MemoryManager` and `OpenAIClient`
- `MemoryManager` interacts with `PersistentStore`
- `UserSession` is managed per API request

---

3. Data Structure Overview

Model	Fields/Attributes	Purpose
UserSession	`session_id`, `user_id`, `history`	Track user context (temp memory)
MemoryRecord	`user_id`, `conversation_history`, `timestamp`	Store permanent memory
ChatRequest	`user_id`, `message`, `session_id`	API input schema
ChatResponse	`response`, `session_id`, `timestamp`	API output schema

---

4. Algorithms/Logic

Chat Flow Pseudocode:

```
def handle_chat_request(user_id, message, session_id):
    session = MemoryManager.get_memory(user_id, session_id)
    prompt = LangChain.build_prompt(session.history, message)
    response = OpenAIClient.generate_response(prompt)
    session.history.append({"user": message, "bot": response})
    MemoryManager.save_memory(user_id, session)
    if end_of_session:
        MemoryManager.persist_memory(user_id, session.history)
    return response
```

---

5. Error Handling

Scenario	Handling Approach
OpenAI API failure	Return error message, log, retry if transient
Data store unavailable	Fallback to temp memory, alert/log
Invalid input (missing fields)	Return 400 Bad Request with error details
Session expired/not found	Create new session, notify user
Unauthorized access	Return 401 Unauthorized

End of Document